

FORMAL DIGITAL FORENSICS INVESTIGATION REPORT

SBT-DF203 — LAB 1: HTTP ANALYSIS USING WIRESHARK – TEXT TRAFFIC

Case Identifier	SBT-DF203-Lab1
Lead Examiner	Nebeuwa Ifeanyichukwu Raphael
Registration / ID No.	202520850LE
Course Code	SBT-DF203
Course Title	Basic Networking Skills for Digital Forensics
Lab Number	Lab 1
Lab Title	HTTP Analysis Using Wireshark - Text Traffic
Date of Submission	September 07, 2026
Primary Target System	Kali Linux Workstation (kali@kali)
Evidence Type	Network Packet Capture (.pcapng) and associated analysis artifacts

Prepared for authorized academic forensic examination

1. Executive Summary

I conducted a controlled digital forensic examination of a local plaintext HTTP transaction generated on a Kali Linux workstation. The examination was performed for SBT-DF203 Lab 1, whose stated scenario required me to create a local webpage containing my name and registration number, capture the resulting HTTP traffic, reconstruct the TCP conversation, identify the HTTP request and response, and document the evidential significance of the relevant protocol layers. The laboratory instructions required the use of an authorized isolated environment and preservation of the original capture while analysis was performed on a verified working copy.

I established the Apache web service on the local host, served the controlled page at 127.0.0.1 over TCP port 80, generated the HTTP request with curl, and captured the traffic on the Linux loopback interface (lo). I preserved the resulting evidence file as evidence/basic.pcapng and created working/basic_working.pcapng for analysis. I verified that the original and working copy produced the identical SHA-256 digest:

48996916527842abf0114489c15dc1448c748af1b4c6a1bc9c3520213cbde502.

My packet-level analysis identified a complete 10-packet TCP conversation (Stream 0), including the SYN, SYN-ACK and ACK handshake, an HTTP GET request for /basic.html, an HTTP 200 OK response from Apache/2.4.68 (Debian), and graceful FIN-ACK termination. Following the TCP stream allowed me to reconstruct the plaintext application payload, which contained the controlled evidence text identifying Ifeanyichukwu Raphael and registration number 202520850LE. I also documented that the loopback capture did not contain normal physical Ethernet MAC addresses, with the observed MAC fields represented as 00:00:00:00:00:00.

Based on the evidence available for this laboratory examination, I determine that the captured HTTP session was successfully acquired, integrity-verified, reconstructed, and documented without evidence of alteration to the preserved capture file. The findings demonstrate the forensic value of plaintext HTTP, TCP connection metadata, application-layer headers, payload reconstruction, and session termination information.

2. Investigation Overview & Scope

2.1 Authority, Scenario and Scope

I conducted this examination within the authorized academic environment specified for SBT-DF203 Lab 1. The source laboratory material identifies the exercise as an individual practical lab in an authorized virtual environment and instructs trainees to use only authorized training evidence. The examination scope therefore remained limited to the locally generated HTTP session and its associated packet-capture evidence.

My examination covered: preparation of the forensic workspace; configuration and verification of Apache; generation of controlled HTTP traffic; packet capture on the loopback interface;

preservation and SHA-256 verification of the capture; TCP handshake analysis; HTTP request/response reconstruction; encapsulation observations; connection termination analysis; and documentation of findings.

2.2 Examination Environment

- Operating System: Kali Linux x86_64.
- User account: kali.
- Working directory: /home/kali/SBT-DF203-Lab1.
- Web server: Apache/2.4.68 (Debian).
- Server port: TCP 80.
- Capture interface: Linux loopback interface lo (127.0.0.1).
- Client observed in the capture: 127.0.0.1:36916.
- Server observed in the capture: 127.0.0.1:80.
- Evidence file: evidence/basic.pcapng.
- Working copy: working/basic_working.pcapng.

3. Forensic Acquisition & Evidence Handling

I first established a dedicated forensic workspace so that original evidence, working copies, exported analysis results, reports, screenshots, and scripts were kept in separate locations. The laboratory procedure specifies the following hierarchy: ~/SBT-DF203-Lab1/{evidence,working,exported,reports,screenshots,scripts}. I used this separation to reduce the risk of accidentally modifying the original capture during analysis.

A terminal window with a dark background and light blue text. The prompt is (kali@kali)-[~]. The user runs 'mkdir -p ~/SBT-DF203-Lab1/{evidence,working,exported,reports,screenshots,scripts}'. The prompt changes to (kali@kali)-[~/SBT-DF203-Lab1]. The user runs 'cd ~/SBT-DF203-Lab1'. The prompt changes to (kali@kali)-[~/SBT-DF203-Lab1]. The user runs 'pwd'. The output is /home/kali/SBT-DF203-Lab1. The user runs 'find . -maxdepth 1 -type d -print'. The output lists the directories: ./working, ./screenshots, ./evidence, ./scripts, ./reports, and ./exported. A large, faint 'KALI' watermark is visible in the background of the terminal window.

```
[sudo] password for kali:
(kali@kali)-[~]
$ mkdir -p ~/SBT-DF203-Lab1/{evidence,working,exported,reports,screenshots,scripts}

(kali@kali)-[~]
$ cd ~/SBT-DF203-Lab1

(kali@kali)-[~/SBT-DF203-Lab1]
$ pwd
/home/kali/SBT-DF203-Lab1

(kali@kali)-[~/SBT-DF203-Lab1]
$ find . -maxdepth 1 -type d -print
.
./working
./screenshots
./evidence
./scripts
./reports
./exported
```

Figure 1: Verification of the forensic workspace directory hierarchy under /home/kali/SBT-DF203-Lab1.

I then prepared the controlled HTTP service. I updated the package repositories, installed Apache2, enabled the Apache service, and created /var/www/html/basic.html containing the laboratory evidence text. I verified that Apache was active and listening on TCP port 80 before generating the traffic.

```
(kali@kali)-[~/SBT-DF203-Lab1]
$ sudo apt update
[sudo] password for kali:
Hit:1 https://repo.protonvpn.com/debian stable InRelease
Get:2 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Packages [21.5 MB]
Get:4 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.5 MB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Packages [113 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [183 kB]
Fetched 75.3 MB in 1min 3s (1,190 kB/s)
1941 packages can be upgraded. Run 'apt list --upgradable' to see them.

(kali@kali)-[~/SBT-DF203-Lab1]
$ sudo systemctl status apache2 --no-pager
o apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; disabled; preset: disabled)
   Active: inactive (dead)
   Docs: https://httpd.apache.org/docs/2.4/

(kali@kali)-[~/SBT-DF203-Lab1]
$
```

Figure 2: Execution of apt update and initial systemctl status apache2 check.

```
(kali@kali)-[~/SBT-DF203-Lab1]
$ sudo apt install -y apache2
The following packages were automatically installed and are no longer required:
  libpostal-data libpostal1
Use 'sudo apt autoremove' to remove them.

Upgrading:
  apache2  apache2-bin  apache2-data  apache2-utils

Summary:
  Upgrading: 4, Installing: 0, Removing: 0, Not Upgrading: 1937
  Download size: 2,025 kB
  Space needed: 3,072 B / 40.7 GB available

Get:1 http://kali.download/kali kali-rolling/main amd64 apache2 amd64 2.4.68-1 [229 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 apache2-bin amd64 2.4.68-1 [1,419 kB]
Get:3 http://kali.download/kali kali-rolling/main amd64 apache2-data all 2.4.68-1 [160 kB]
Get:4 http://kali.download/kali kali-rolling/main amd64 apache2-utils amd64 2.4.68-1 [217 kB]
Fetched 2,025 kB in 12s (163 kB/s)
(Reading database... 426373 files and directories currently installed.)
Preparing to unpack .../apache2_2.4.68-1_amd64.deb...
Unpacking apache2 (2.4.68-1) over (2.4.66-8)...
Preparing to unpack .../apache2-bin_2.4.68-1_amd64.deb...
Unpacking apache2-bin (2.4.68-1) over (2.4.66-8)...
Preparing to unpack .../apache2-data_2.4.68-1_all.deb...
Unpacking apache2-data (2.4.68-1) over (2.4.66-8)...
Preparing to unpack .../apache2-utils_2.4.68-1_amd64.deb...
Unpacking apache2-utils (2.4.68-1) over (2.4.66-8)...
```

Figure 3: Installation and configuration of the Apache2 web server components.


```

(kali@kali)-[~/SBT-DF203-Lab1]
$ sudo systemctl enable --now apache2
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2
Created symlink '/etc/systemd/system/multi-user.target.wants/apache2.service' -> '/usr/lib/systemd/system/apache2.service'.

(kali@kali)-[~/SBT-DF203-Lab1]
$

```

Figure 4: Enabling the Apache2 service through systemd.

```

(kali@kali)-[~/SBT-DF203-Lab1]
$ printf '<!DOCTYPE html>\n<html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: Ifeanyichukwu Raphael</p><p>Reg No: 202520850LE</p></body></html>\n' | sudo tee /var/www/html/basic.html
<!DOCTYPE html>
<html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: Ifeanyichukwu Raphael</p><p>Reg No: 202520850LE</p></body></html>

```

Figure 5: Generation of basic.html containing the examiner name and registration number.

```

Session Actions Edit View Help
(kali@kali)-[~/SBT-DF203-Lab1]
$ sudo systemctl status apache2 --no-pager
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-09-07 10:02:03 EDT; 1min 12s ago
     Invocation: c92d47ed28bf4faeba4482f3777bef2f
       Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 42526 (apache2)
      Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
      Tasks: 6 (limit: 2075)
     Memory: 36.9M (peak: 36.9M)
        CPU: 432ms
      CGroup: /system.slice/apache2.service
              └─42526 /usr/sbin/apache2 -k start -DFOREGROUND
                └─42566 /usr/sbin/apache2 -k start -DFOREGROUND
                  └─42567 /usr/sbin/apache2 -k start -DFOREGROUND
                    └─42568 /usr/sbin/apache2 -k start -DFOREGROUND
                      └─42569 /usr/sbin/apache2 -k start -DFOREGROUND
                        └─42570 /usr/sbin/apache2 -k start -DFOREGROUND

Sep 07 10:02:01 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
Sep 07 10:02:02 kali apachectl[42526]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, please see the /usr/sbin/httpd.conf file and uncomment the 'ServerName' line for being the recommended way to deal with this situation.
Sep 07 10:02:03 kali systemd[1]: Started apache2.service - The Apache HTTP Server.
Hint: Some lines were ellipsized, use -l to show in full.

(kali@kali)-[~/SBT-DF203-Lab1]
$ sudo ss -ltnp | grep ':80'
LISTEN 0      511          *:*          *:*          users:((("apache2",pid=42527,fd=4),("apache2",pid=42569,fd=4),("apache2",pid=42568,fd=4),("apache2",pid=42567,fd=4),("apache2",pid=42566,fd=4),("apache2",pid=42526,fd=4)))

```

Figure 6: Confirmation of the active Apache process and listening state on TCP port 80.

I confirmed that the controlled page was accessible locally and generated a verbose HTTP request using curl. This provided an independent application-level confirmation that the server was returning the expected resource before and during the capture process.

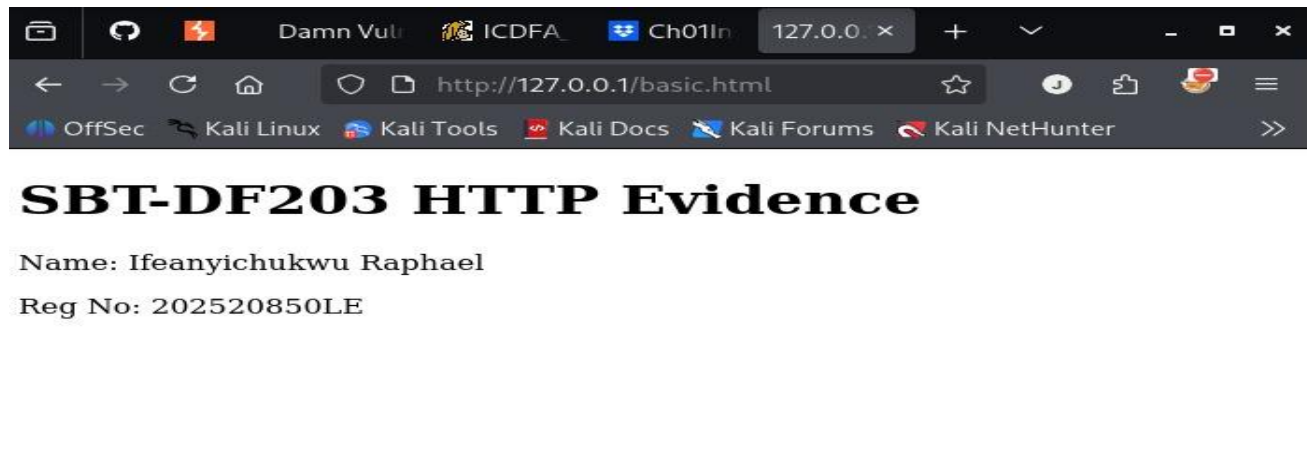


Figure 7: Rendering of the controlled HTML document displaying the examiner name and registration number.

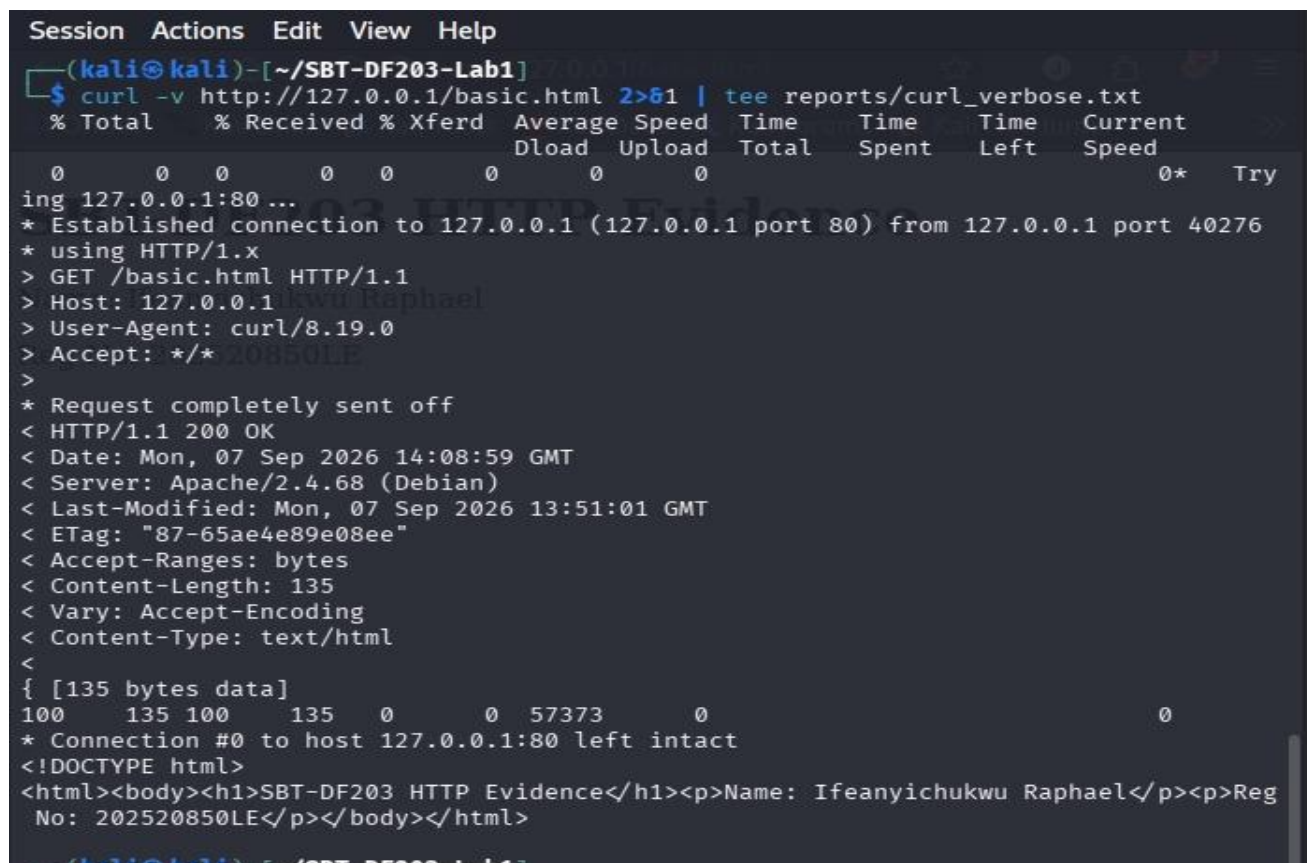
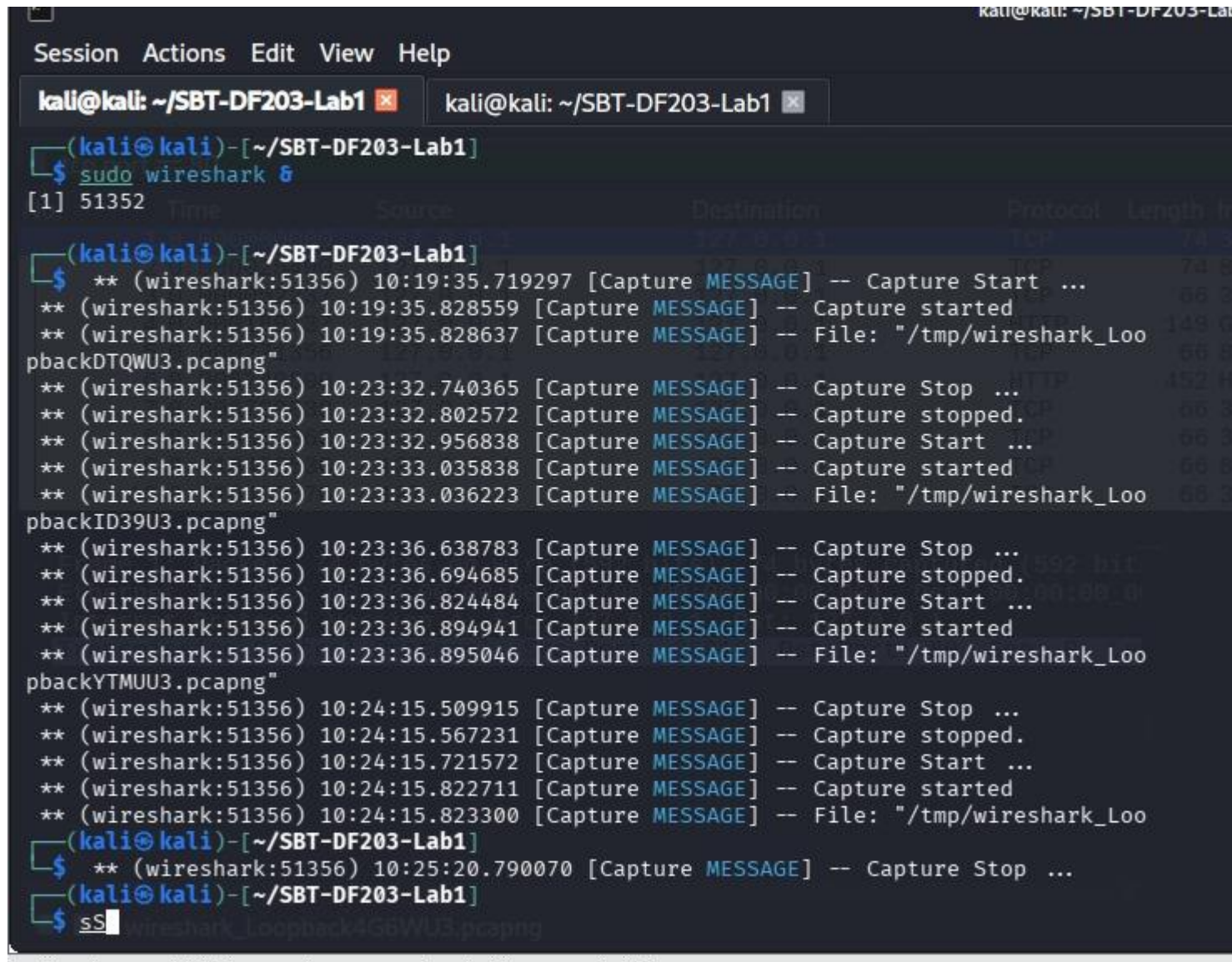


Figure 8: Execution of curl -v against the controlled page and observation of the HTTP request/response headers.

For acquisition, the laboratory procedure permits either Wireshark or tshark. I captured traffic on lo using the TCP port 80 capture filter and saved the resulting packet capture as evidence/basic.pcapng. After capture completion, I created a working copy using cp --preserve=timestamps. I retained the original evidence file separately and conducted subsequent packet analysis against the working copy.



```
kali@kali: ~/SBT-DF203-Lab1
Session Actions Edit View Help
kali@kali: ~/SBT-DF203-Lab1
(kali@kali)-[~/SBT-DF203-Lab1]
$ sudo wireshark &
[1] 51352
(kali@kali)-[~/SBT-DF203-Lab1]
$ ** (wireshark:51356) 10:19:35.719297 [Capture MESSAGE] -- Capture Start ...
** (wireshark:51356) 10:19:35.828559 [Capture MESSAGE] -- Capture started
** (wireshark:51356) 10:19:35.828637 [Capture MESSAGE] -- File: "/tmp/wireshark_LoopbackDTQWU3.pcapng"
** (wireshark:51356) 10:23:32.740365 [Capture MESSAGE] -- Capture Stop ...
** (wireshark:51356) 10:23:32.802572 [Capture MESSAGE] -- Capture stopped.
** (wireshark:51356) 10:23:32.956838 [Capture MESSAGE] -- Capture Start ...
** (wireshark:51356) 10:23:33.035838 [Capture MESSAGE] -- Capture started
** (wireshark:51356) 10:23:33.036223 [Capture MESSAGE] -- File: "/tmp/wireshark_LoopbackID39U3.pcapng"
** (wireshark:51356) 10:23:36.638783 [Capture MESSAGE] -- Capture Stop ...
** (wireshark:51356) 10:23:36.694685 [Capture MESSAGE] -- Capture stopped.
** (wireshark:51356) 10:23:36.824484 [Capture MESSAGE] -- Capture Start ...
** (wireshark:51356) 10:23:36.894941 [Capture MESSAGE] -- Capture started
** (wireshark:51356) 10:23:36.895046 [Capture MESSAGE] -- File: "/tmp/wireshark_LoopbackYTMUU3.pcapng"
** (wireshark:51356) 10:24:15.509915 [Capture MESSAGE] -- Capture Stop ...
** (wireshark:51356) 10:24:15.567231 [Capture MESSAGE] -- Capture stopped.
** (wireshark:51356) 10:24:15.721572 [Capture MESSAGE] -- Capture Start ...
** (wireshark:51356) 10:24:15.822711 [Capture MESSAGE] -- Capture started
** (wireshark:51356) 10:24:15.823300 [Capture MESSAGE] -- File: "/tmp/wireshark_Loopback4G6WU3.pcapng"
(kali@kali)-[~/SBT-DF203-Lab1]
$ ** (wireshark:51356) 10:25:20.790070 [Capture MESSAGE] -- Capture Stop ...
(kali@kali)-[~/SBT-DF203-Lab1]
$ ss
```

Figure 9: Capture of the controlled HTTP session on interface lo before and during the curl request.

4. Data Integrity & Verification

I treated the original packet capture as the primary evidence object and used a working copy for analysis. To establish that the working copy was an exact byte-for-byte duplicate of the preserved

evidence file, I calculated SHA-256 hashes for both files. The laboratory instructions specifically require preservation of original capture files and analysis of verified working copies.

```
(kali@kali)-[~/SBT-DF203-Lab1]
$ cp --preserve=timestamps evidence/basic.pcapng working/basic_working.pcapng

(kali@kali)-[~/SBT-DF203-Lab1]
$ sha256sum evidence/basic.pcapng working/basic_working.pcapng | tee reports/capture_hashes.txt
48996916527842abf0114489c15dc1448c748af1b4c6a1bc9c3520213cbde502 evidence/basic.pcapng
48996916527842abf0114489c15dc1448c748af1b4c6a1bc9c3520213cbde502 working/basic_working.pcapng

(kali@kali)-[~/SBT-DF203-Lab1]
```

Figure 10: Generation and comparison of SHA-256 digests for the original evidence and working copy.

The recorded SHA-256 values were identical:

```
48996916527842abf0114489c15dc1448c748af1b4c6a1bc9c3520213cbde502
evidence/basic.pcapng

48996916527842abf0114489c15dc1448c748af1b4c6a1bc9c3520213cbde502
working/basic_working.pcapng
```

Because both files produced the same SHA-256 digest, I verified that the working copy was cryptographically consistent with the preserved evidence file at the time of comparison. Within the limits of this verification, there was no detected bit-level alteration introduced by the duplication or subsequent handling represented by these hash values.

5. Forensic Analysis & Artifact Recovery

5.1 TCP Session Identification and Three-Way Handshake

I opened working/basic_working.pcapng in Wireshark and isolated TCP Stream 0. The capture contained exactly 10 packets associated with the observed HTTP session. I examined the first three packets to establish the TCP connection state.

- Frame 1 — SYN: the client at 127.0.0.1:36916 initiated a TCP connection toward 127.0.0.1:80. The relative sequence number was 0.
- Frame 2 — SYN, ACK: the server acknowledged the client's SYN with Ack=1 and supplied its own relative sequence number of 0.
- Frame 3 — ACK: the client acknowledged the server's SYN with Seq=1 and Ack=1, completing the three-way handshake and establishing the TCP connection.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	TCP	74	36916 → 80 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=146...
2	0.000025490	127.0.0.1	127.0.0.1	TCP	74	80 → 36916 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PER...
3	0.000052237	127.0.0.1	127.0.0.1	TCP	66	36916 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=1463044614 TSecr=...
4	0.000189737	127.0.0.1	127.0.0.1	HTTP	149	GET /basic.html HTTP/1.1
5	0.000271356	127.0.0.1	127.0.0.1	TCP	66	80 → 36916 [ACK] Seq=1 Ack=84 Win=65536 Len=0 TSval=1463044614 TSecr=...
6	0.000740690	127.0.0.1	127.0.0.1	HTTP	452	HTTP/1.1 200 OK (text/html)
7	0.000799038	127.0.0.1	127.0.0.1	TCP	66	36916 → 80 [ACK] Seq=84 Ack=387 Win=65280 Len=0 TSval=1463044621 TSe...
8	0.010302363	127.0.0.1	127.0.0.1	TCP	66	36916 → 80 [FIN, ACK] Seq=84 Ack=387 Win=65536 Len=0 TSval=146304462...
9	0.010408536	127.0.0.1	127.0.0.1	TCP	66	80 → 36916 [FIN, ACK] Seq=387 Ack=85 Win=65536 Len=0 TSval=146304462...
10	0.010462870	127.0.0.1	127.0.0.1	TCP	66	36916 → 80 [ACK] Seq=85 Ack=388 Win=65536 Len=0 TSval=1463044624 TSe...

Frame 1: Packet, 74 bytes on wire (592 bits), 74 bytes captured (592 bit...
 Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_0...
 Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
 Transmission Control Protocol, Src Port: 36916, Dst Port: 80, Seq: 0, Le...

wireshark_Loopback4G6WU3.pcapng Packets: 10 · Displayed: 10 (100.0%) · Dropped: 0 (0.0%) · Profile: Default

Figure 11: The complete 10-packet flow associated with the loopback HTTP transaction.

5.2 HTTP Request and Response Reconstruction

I used tshark to extract the application-layer metadata from the working capture and then used Wireshark's Follow TCP Stream function to reconstruct the complete application conversation.

The HTTP request identified in Frame 4 contained the following forensic values:

- Method: GET
- Request URI: /basic.html
- Host: 127.0.0.1
- User-Agent: curl/8.19.0

The HTTP response identified in Frame 6 contained:

- Status: 200 OK
- Server: Apache/2.4.68 (Debian)
- Content-Type: text/html
- Content-Length: 135 bytes

```

(kali㉿kali)-[~/SBT-DF203-Lab1]
$ tshark -r working/basic_working.pcapng -Y 'http.request' -T fields \ Length Info
-e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e http.request.method -e http.request.uri -e http.host -e http.user_agent \
| tee reports/http_requests.tsv
4      2026-09-07T10:24:38.205521843-0400      127.0.0.1      36916      127.0.0.1      80
basic.html      127.0.0.1      curl/8.19.0

(kali㉿kali)-[~/SBT-DF203-Lab1]
$ tshark -r working/basic_working.pcapng -Y 'http.response' -T fields \
-e frame.number -e frame.time -e http.response.code -e http.response.phrase \
-e http.server -e http.content_type -e http.content_length \
| tee reports/http_responses.tsv
6      2026-09-07T10:24:38.212072796-0400      200      OK      Apache/2.4.68 (Debian) text
/html      135

(kali㉿kali)-[~/SBT-DF203-Lab1]
$

```

Figure 12: Command-line extraction of HTTP metadata showing the request fields and the 200 OK response parameters.

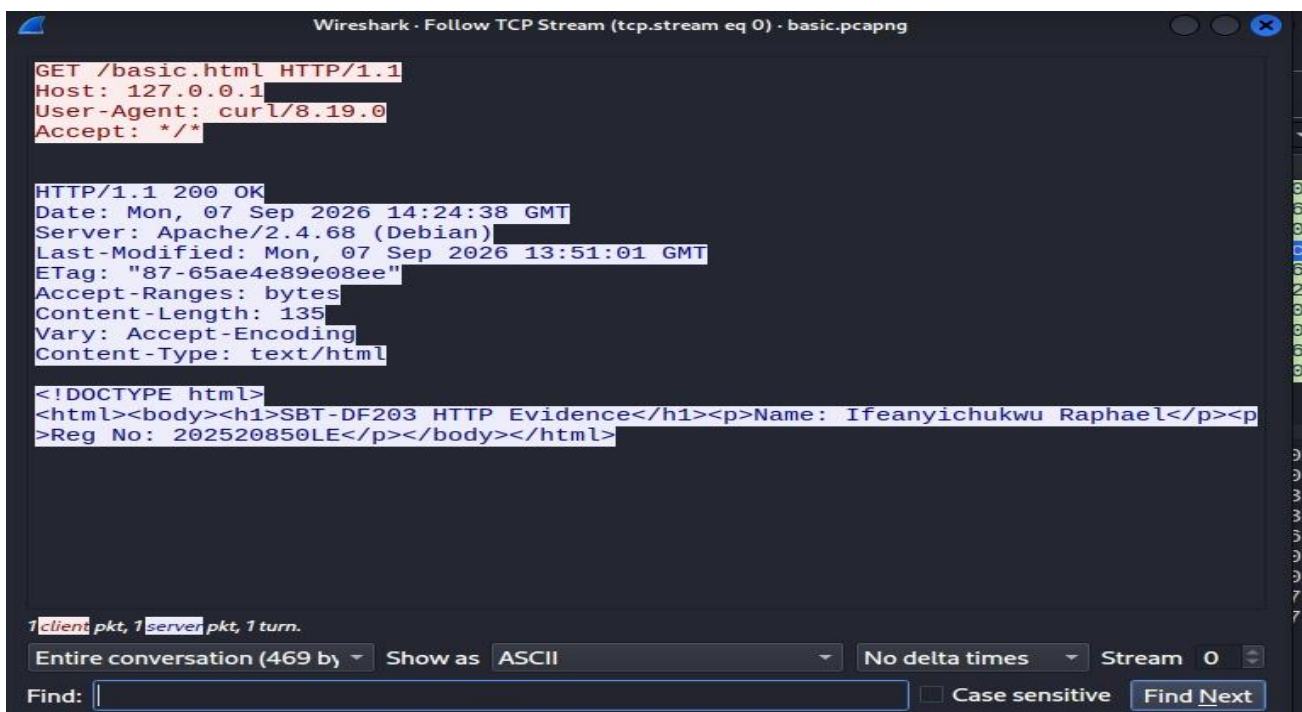


Figure 13: Reconstructed ASCII TCP stream showing the plaintext HTTP request and server response payload.

The reconstructed application payload contained the controlled laboratory evidence text:

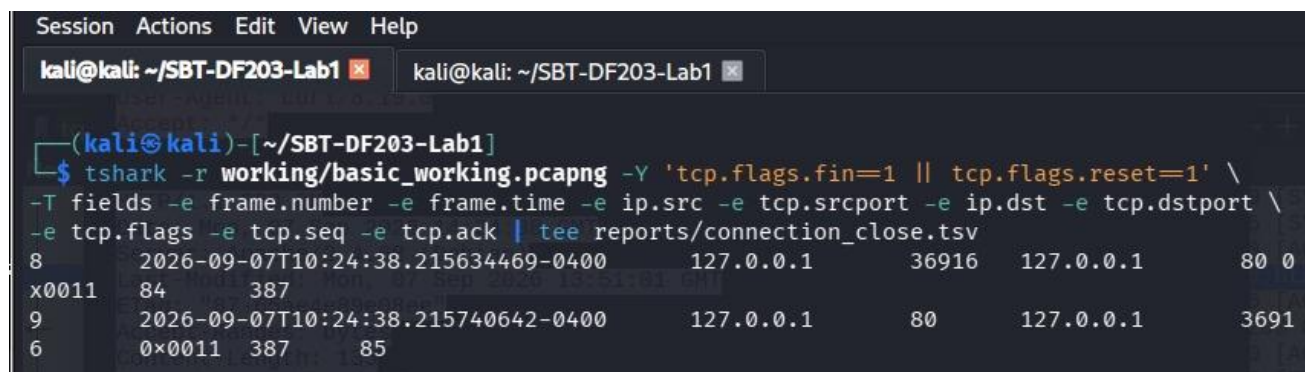
<!DOCTYPE html><html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: Ifeanyichukwu Raphael</p><p>Reg No: 202520850LE</p></body></html>. The recovery of this content

demonstrates that the HTTP application data was transmitted in plaintext and could be reconstructed directly from the packet stream.

5.3 Encapsulation, Loopback and Connection Teardown

I examined the HTTP transaction through the Frame, Internet Protocol, Transmission Control Protocol and Hypertext Transfer Protocol layers. This allowed me to correlate the captured packet structure with the application request and response. The laboratory specifically requires examination of the relationship between frame length, IP total length, TCP header length and TCP payload length; the supplied examination results establish the HTTP/TCP session but do not provide a complete numerical layer-length table, so I have not introduced unsupported measurements.

I then filtered the capture for TCP FIN or RESET flags to determine how the session ended.



```
Session Actions Edit View Help
kali@kali: ~/SBT-DF203-Lab1 x kali@kali: ~/SBT-DF203-Lab1 x

(kali@kali)-[~/SBT-DF203-Lab1]
$ tshark -r working/basic_working.pcapng -Y 'tcp.flags.fin==1 || tcp.flags.reset==1' \
-T fields -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e tcp.flags -e tcp.seq -e tcp.ack | tee reports/connection_close.tsv
8      2026-09-07T10:24:38.215634469-0400      127.0.0.1      36916      127.0.0.1      80 0
x0011  84      387
9      2026-09-07T10:24:38.215740642-0400      127.0.0.1      80      127.0.0.1      3691
6      0x0011  387      85
```

Figure 14: Filtering packets containing FIN flags to document the observed session closure.

Frames 8–10 showed graceful FIN-ACK termination. I therefore determine that the observed TCP conversation closed through an orderly TCP teardown rather than an observed reset.

The capture was performed on the Linux loopback interface. The observed source and destination MAC fields were 00:00:00:00:00:00. This is consistent with the laboratory's stated loopback behavior: the traffic remained within the host's network stack and did not traverse a physical Ethernet network interface. Consequently, normal physical source and destination MAC addresses were not available as Ethernet evidence for this local session. The laboratory notes that a two-VM host-only capture would be required where actual Ethernet MAC evidence is needed.

6. Chronological Forensic Timeline

Stage	Packet/Artifact	Observed Value	Forensic Significance
Connection initiation	Frame 1	SYN, 127.0.0.1:36916 → 127.0.0.1:80	Client requests TCP session.
Connection establishment	Frames 2–3	SYN-ACK followed by ACK	TCP session becomes established.
HTTP request	Frame 4	GET /basic.html; Host	Identifies requested

		127.0.0.1; curl/8.19.0	resource and client metadata.
HTTP response	Frame 6	200 OK; Apache/2.4.68; text/html; 135 bytes	Confirms successful server response and payload characteristics.
Payload recovery	Follow TCP Stream	Name: Ifeanyichukwu Raphael; Reg No: 202520850LE	Recovers plaintext application content.
Session closure	Frames 8–10	FIN-ACK sequence	Documents graceful connection termination.
Integrity verification	Evidence + working copy	Identical SHA-256 digest	Supports integrity of the working copy.

7. Findings & Evidence Matrix

Finding Parameter	Recorded Forensic Value
Evidence File Name	evidence/basic.pcapng
SHA-256 Digest	48996916527842abf0114489c15dc1448c748af1b4c6a1bc9c3520213cbde502
Client Socket	127.0.0.1:36916
Server Socket	127.0.0.1:80
Transport Protocol	TCP (Stream 0)
Application Protocol	HTTP/1.1
Packet Count	10 packets in the identified session
HTTP Request	GET /basic.html
HTTP Response	200 OK
Server	Apache/2.4.68 (Debian)
Content Type	text/html
Content Length	135 bytes
Recovered Identity	Ifeanyichukwu Raphael Reg No: 202520850LE
Termination State	Graceful FIN-ACK teardown (Frames 8–10)
MAC Observation	00:00:00:00:00:00 on loopback capture

8. Limitations

I confined my conclusions to the evidence and observations documented in the supplied laboratory materials. The examination did not establish physical Ethernet MAC addresses because the traffic

was captured on the loopback interface. The supplied evidence also does not provide a complete numerical record of every frame/IP/TCP length relationship, so I have not inferred values that were not documented. Finally, the laboratory scenario is a controlled local HTTP session and should not be interpreted as evidence of activity on a third-party or production system.

9. Conclusion & Final Forensic Determination

I successfully completed the documented forensic workflow for the SBT-DF203 HTTP analysis exercise. I prepared an isolated workspace, configured the local Apache service, generated controlled plaintext HTTP traffic, preserved the resulting PCAPNG evidence, created a working copy, and verified the copy against the original using SHA-256.

My analysis established a complete TCP session consisting of the expected SYN, SYN-ACK and ACK handshake, followed by an HTTP GET request and a successful HTTP 200 OK response. I reconstructed the TCP stream and recovered the plaintext HTML payload containing the controlled identity information. I also documented the graceful FIN-ACK termination and the absence of normal physical Ethernet MAC addresses in the loopback capture.

The final forensic determination is that the captured evidence reliably documents the controlled local HTTP transaction and, based on the matching SHA-256 values, the analyzed working copy remained cryptographically consistent with the preserved evidence file. The examination demonstrates that unencrypted HTTP exposes application-layer request metadata and content in a form that can be recovered directly from packet captures.

10. Appendix A - Commands and Processing Record

```
mkdir -p ~/SBT-DF203-  
Lab1/{evidence,working,exported,reports,screenshots,scripts}
```

```
sudo apt update
```

```
sudo apt install -y apache2 curl wireshark tshark
```

```
sudo systemctl enable --now apache2
```

```
curl -v http://127.0.0.1/basic.html
```

```
sudo ss -lntp | grep ':80'
```

```
sudo tshark -i lo -f 'tcp port 80' -w evidence/basic.pcapng
```

```
cp --preserve=timestamps evidence/basic.pcapng  
working/basic_working.pcapng
```

```
sha256sum evidence/basic.pcapng working/basic_working.pcapng | tee  
reports/capture_hashes.txt
```

```
tshark -r working/basic_working.pcapng -Y 'tcp.stream eq 0 &&  
tcp.flags.syn==1'
```

```
tshark -r working/basic_working.pcapng -Y 'http.request'
```

```
tshark -r working/basic_working.pcapng -Y 'http.response'
```

```
tshark -r working/basic_working.pcapng -Y 'tcp.flags.fin==1 ||  
tcp.flags.reset==1'
```

11. Appendix B - Chain-of-Custody Record

Event	Item / Location	Action	Integrity / Notes
1	Generated local capture	Saved as evidence/basic.pcapng	Original evidence retained.
2	Evidence preservation	Copied to working/basic_working.pcapng	cp --preserve=timestamps used.
3	Integrity verification	SHA-256 calculated for both files	Both values matched exactly.
4	Forensic analysis	Wireshark/tshark analysis of working copy	Original not used as analysis target.
5	Reporting	Findings and screenshots documented	Evidence references retained in report.

12. Examiner Declaration

I confirm that this report documents the forensic procedures, observations and conclusions supported by the SBT-DF203 Lab 1 evidence supplied for this examination. I conducted the analysis within the authorized training environment and have distinguished documented